

Version Control & Git

What is Version Control?

Version control systems (VCSs) are tools used to track changes to source code (or other collections of files and folders).

- Maintain a history of changes.
- Facilitate collaboration.
- Track changes in a series of **snapshots**.

Why is it useful?

Even when you're working by yourself, it can let you:

- look at old snapshots of a project,
- keep a log of why certain changes were made,
- work on parallel branches of development.

When working with others, it's an invaluable tool for:

- seeing what other people have changed,
- resolving conflicts in concurrent development.

Git

While other VCSs exist, **Git** is the de facto standard for version control.



Centralized vs. Distributed VCS

- **Centralized VCS (CVCS):** Uses a single central repository. Developers check out files from the central server and commit changes back to it. (e.g., Subversion, Perforce).
- **Distributed VCS (DVCS):** Every developer has a full copy of the repository, including its entire history. Changes can be committed locally and then "pushed" to other repositories. (e.g., Git, Mercurial).

Source of Truth

A critical concept in version control is the "source of truth." This is the single, authoritative location for a project's code.

- In a **Centralized VCS**, the source of truth is baked into the system: it's the central server.
- In a **Distributed VCS** like Git, there is no inherent source of truth. It is a matter of **policy**.
- Most projects using DVCS designate a "main" repository (e.g., on GitHub) as the conceptual source of truth.

Branch Management and The "One-Version" Rule

A key policy for managing a source of truth is **trunk-based development**.

- All developers work on a single branch, `master` or `trunk`.
- Development branches are short-lived.

This leads to the **One-Version Rule**:

Developers must never have a choice of "What version of this component should I depend upon?"

This rule simplifies dependency management and avoids "dependency hell."

Git's data model

Git's ingenuity is in its well-thought-out data model that enables all the nice features of version control. Git's interface may be confusing and ugly, but its data model is simple and elegant. We will learn Git from the bottom-up, starting with its data model.

Snapshots

Git models the history of a collection of files and folders as a series of **snapshots**.

- A file is a **blob** (a bunch of bytes).
- A directory is a **tree** (maps names to blobs or trees).
- A snapshot is the top-level tree that is being tracked.

```
<root> (tree)
|
+- foo (tree)
|   |
|   + bar.txt (blob, contents = "hello world")
|
+- baz.txt (blob, contents = "git is wonderful")
```

Modeling history: relating snapshots

In Git, a history is a **directed acyclic graph (DAG)** of snapshots.

Each snapshot refers to a set of **parents**, the snapshots that preceded it.

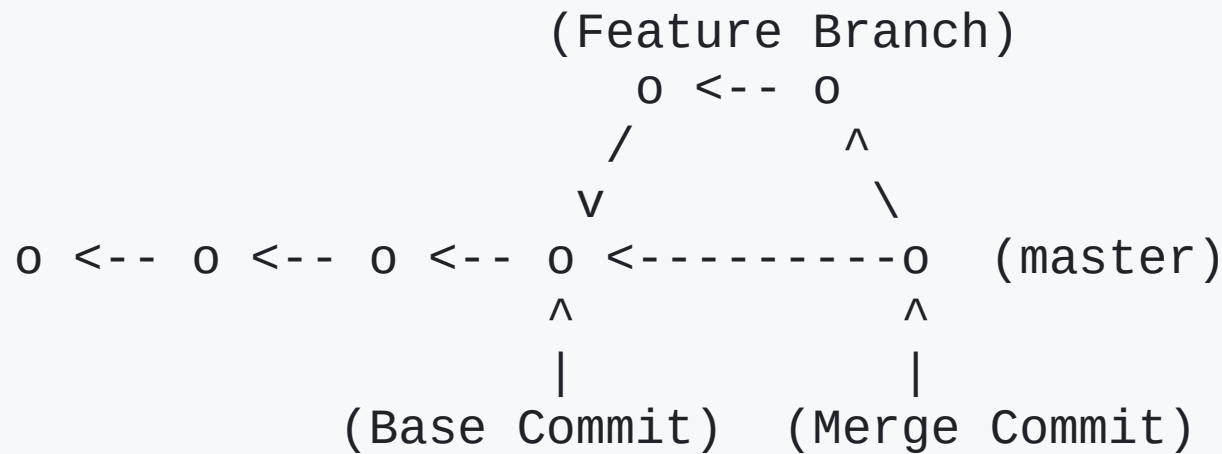
Git calls these snapshots **commits**. The "o" in the diagram below represents a commit, and the arrows point to its parent(s).

```
o <-- o <-- o <-- o (master)
      ^
      \
        --- o <-- o (feature)
```

The arrows point to the parent of each commit.

Commits

A commit can have multiple parents, such as when merging two feature branches into a main branch.



Commits in Git are **immutable**. Any **edits** to commit history create entirely new commits rather than altering existing ones.

Data model, as pseudocode

```
// a file is a bunch of bytes
class Blob { byte[] contents; }

// A directory contains named files and directories
class Tree { Map<String, Object> entries; } // Where Object can be Blob or Tree

// A commit has parents, metadata, and the top-level tree
class Commit {
    List<Commit> parents; // List of pointers to parent commits
    String author;
    String message;
    Tree snapshot; // Pointer to the top-level tree of the snapshot
}
```

Objects and content-addressing

An “object” is a blob, tree, or commit.

```
interface GitObject {} // Represents a common interface for Blob, Tree, and Commit

// In Git's data store, all objects are content-addressed by their SHA-1 hash.
class ObjectStore {
    Map<String, GitObject> objects; // String is the SHA-1 hash of the object

    public void store(GitObject object) {
        String id = sha1(object); // Assume sha1 is a utility function
        objects.put(id, object);
    }

    public GitObject load(String id) {
        return objects.get(id);
    }
}
```

Objects reference other objects by their hash.

References

References are human-readable names for SHA-1 hashes. They are pointers to commits and are mutable.

```
class ReferenceStore {
    Map<String, String> references; // String is the name of the reference, String is the SHA-1 hash

    public void updateReference(String name, String id) {
        references.put(name, id);
    }

    public String readReference(String name) {
        return references.get(name);
    }

    public GitObject loadReference(String nameOrId, ObjectStore objectStore) { // Assuming ObjectStore is available
        if (references.containsKey(nameOrId)) {
            return objectStore.load(references.get(nameOrId));
        } else {
            return objectStore.load(nameOrId); // Try loading directly by ID if not a named reference
        }
    }
}
```

`master` is a reference that usually points to the latest commit in the main branch. `HEAD` is a special reference to the current snapshot.

Repositories

A Git repository is the data `objects` and `references` .

On disk, all Git stores are objects and references. All `git` commands map to some manipulation of the commit DAG by adding objects and adding/updating references.

Staging area

The staging area is where you can specify which modifications should be included in the next snapshot. This allows you to build clean, atomic commits.

- You can implement two features and create two separate commits.
- You can commit a bugfix while discarding debugging print statements.

Git command-line interface

Basics

- `git help <command>` : get help for a git command
- `git init` : creates a new git repo
- `git status` : tells you what's going on
- `git add <filename>` : adds files to staging area
- `git commit` : creates a new commit
- `git log` : shows a flattened log of history
- `git log --all --graph --decorate` : visualizes history as a DAG
- `git diff <filename>` : show changes relative to the staging area
- `git diff <revision> <filename>` : shows differences between snapshots
- `git checkout <revision>` : updates HEAD

Branching and merging

- `git branch` : shows branches
- `git branch <name>` : creates a branch
- `git switch <name>` : switches to a branch
- `git checkout -b <name>` : creates a branch and switches to it
- `git merge <revision>` : merges into current branch
- `git mergetool` : use a fancy tool to help resolve merge conflicts
- `git rebase` : rebase set of patches onto a new base

Remotes

- `git remote` : list remotes
- `git remote add <name> <url>` : add a remote
- `git push <remote> <local branch>:<remote branch>` : send objects to remote
- `git fetch` : retrieve objects/references from a remote
- `git pull` : same as `git fetch`; `git merge`
- `git clone` : download repository from remote

Undo

- `git commit --amend` : edit a commit's contents/message
- `git reset <file>` : unstage a file
- `git restore` : discard changes

Advanced Git

- `git config` : Git is highly customizable
- `git add -p` : interactive staging
- `git rebase -i` : interactive rebasing
- `git blame` : show who last edited which line
- `git stash` : temporarily remove modifications
- `git bisect` : binary search history for regressions
- `git revert` : create a new commit that reverses an earlier commit
- `.gitignore` : specify intentionally untracked files to ignore

GitHub

GitHub is a web-based platform for version control and collaboration, built around Git.

- **Repository Hosting:** Provides cloud storage for Git repositories.
- **Collaboration Features:** Facilitates teamwork through features like Pull Requests, Issues, and Code Reviews.
- **Social Coding:** Enables developers to discover, contribute to, and manage open-source projects.
- **Not Git itself:** Git is the underlying version control system, while GitHub is a service that hosts Git repositories and adds features.

We will be using GitHub and more specifically GitHub Classroom to submit assignments and manage projects.

[GitHub](#)

[GitHub Classroom](#)

GitHub Desktop

GitHub Desktop is a free and open-source graphical user interface (GUI) for Git.

- **Simplifies Git Workflow:** Provides a visual way to interact with Git and GitHub repositories without using the command line.
- **Common Tasks:** Supports common Git operations like committing, branching, merging, and pull requests through a user-friendly interface.
- **Local Development:** Integrates with your local development environment, making it easier to clone repositories, make changes, and synchronize with GitHub.

[GitHub Desktop](#)

Resources

- [Pro Git](#) - Highly recommended reading
- [Oh Shit, Git!?!](#)
- [Git for Computer Scientists](#)
- [Learn Git Branching](#)